12 OPERATOR OVERLOADING • DETAILED SOLUTIONS

output solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Predict the Output & Debugging (Module 12: Operator Overloading)

Solution to Q7

- `a + b + a` associates Left-to-Right as `(a + b) + a`.
- `a + b` produces `complex_system(2+3, 5+1) = (5, 6)`.
- `(5, 6) + a` produces `complex_system(5+2, 6+5) = (7, 11)`.
- **Guaranteed Output**:

```text
7+11i
```

Solution to Q8

- `b` starts with `val = 10`.
- `box b1 = ++b;` calls prefix `operator++()`: `b.val` becomes `15`, and `b1.val` is `15`.
- `box b2 = b++;` calls postfix `operator++(int)`: `b2` receives the old copy (`15`), while `b.val` becomes `15 + 10 = 25`.
- **Guaranteed Output**:

```text
15 15 25
```

Solution to Q9 (Debugging Mutating `operator+`)

- **Bug**: `real = real + obj.real;` modifies `this→real` (`c1.real`) directly! Binary `+` should never mutate its left operand `c1`.
- **Fix**: Construct and return a temporary `complex_system` object and mark the member function `const`:

```cpp
complex_system operator+(const complex_system& obj) const {
  return complex_system(real + obj.real, imag + obj.imag);
}
```

Solution to Q10 (Debugging `5 + c1`)

- **Bug**: Member `operator+` requires the left operand to be of type `complex_system` (`c1.operator+(5)`), so `5 + c1` fails because `5` is a built-in `int`.

- **Fix**: Declare `operator+` as a `friend` function taking two `const complex_system&` parameters (since `complex_system(int r = 0, int i = 0)` is a non-explicit converting constructor, `5` implicitly converts to `complex_system(5, 0)`).